

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC KHOA HỌC**

~~~~~\*~~~~~

**KHOA CÔNG NGHỆ THÔNG TIN**



**BÀI TIỂU LUẬN**

**Phát Triển Ứng Dụng IOT**

**ĐỀ TÀI:**

Giám sát chất lượng không khí với ESP32 và cảm biến MQ  
Đo nồng độ khí CO, bụi PM2.5 bằng cảm biến MQ-135 hoặc PMS5003

***Giáo viên hướng dẫn : Võ Việt Dũng***

**Huế, 05/04/2026**

# DANH MỤC CÁC TỪ VIẾT TẮT

| Từ viết tắt | Giải thích                                                                                |
|-------------|-------------------------------------------------------------------------------------------|
| CO          | Carbon Monoxide (Khí Carbon Monoxide)                                                     |
| ESP32       | Vi điều khiển hỗ trợ kết nối Wi-Fi và Bluetooth                                           |
| HTTP        | Hypertext Transfer Protocol (Giao thức truyền tải siêu văn bản)                           |
| IDE         | Integrated Development Environment (Môi trường phát triển tích hợp)                       |
| IoT         | Internet of Things (Internet vạn vật)                                                     |
| PM2.5       | Particulate Matter 2.5 (Bụi mịn có đường kính khí động học $\leq 2.5 \mu\text{m}$ )       |
| PPM         | Parts Per Million (Đơn vị đo phần triệu)                                                  |
| UART        | Universal Asynchronous Receiver-Transmitter (Giao thức truyền thông nối tiếp bất đồng bộ) |
| Web UI      | Web User Interface (Giao diện người dùng trên nền tảng Web)                               |
| ADC         | Analog-to-Digital Converter (Bộ chuyển đổi tương tự - số)                                 |
| AJAX        | Asynchronous JavaScript and XML                                                           |
| JSON        | JavaScript Object Notation (Định dạng dữ liệu trao đổi)                                   |
| LAN         | Local Area Network (Mạng máy tính cục bộ)                                                 |
| MQTT        | Message Queuing Telemetry Transport (Giao thức nhắn tin IoT)                              |
| SoC         | System on a Chip (Hệ thống trên một vi mạch)                                              |

# MỤC LỤC

|                                                       |    |
|-------------------------------------------------------|----|
| 1. Lý do chọn đề tài .....                            | 4  |
| 2. Mục tiêu nghiên cứu .....                          | 4  |
| 3. Đối tượng và phạm vi nghiên cứu .....              | 4  |
| CHƯƠNG 1: CƠ SỞ LÝ THUYẾT .....                       | 5  |
| 1.1. Tổng quan về Internet of Things (IoT) .....      | 5  |
| 1.2. Vi điều khiển ESP32 .....                        | 5  |
| 1.3. Cảm biến chất lượng không khí .....              | 6  |
| 1.3.1. Cảm biến khí MQ-135 .....                      | 6  |
| 1.3.2. Cảm biến bụi mịn PMS5003 .....                 | 6  |
| 1.4. Web Server nhúng trên ESP32 .....                | 7  |
| 1.5. Wokwi và lý do sử dụng Slide Potentiometer ..... | 7  |
| CHƯƠNG 2: THIẾT KẾ HỆ THỐNG .....                     | 8  |
| 2.1. Yêu cầu thiết kế .....                           | 8  |
| 2.2. Sơ đồ khối hệ thống .....                        | 8  |
| 2.3. Sơ đồ đấu nối .....                              | 8  |
| 2.3.1. Phần cứng thực tế .....                        | 8  |
| 2.3.2. Mô phỏng Wokwi .....                           | 9  |
| 2.4. Lưu đồ thuật toán .....                          | 10 |
| CHƯƠNG 3: TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ .....        | 10 |
| 3.1. Môi trường phát triển .....                      | 10 |
| 3.2. Mã nguồn và giải thuật .....                     | 11 |
| 3.2.1. Đọc cảm biến và ánh xạ giá trị .....           | 11 |
| 3.2.2. Ngưỡng cảnh báo .....                          | 11 |
| 3.2.3. Giao diện Web và Telegram Bot .....            | 11 |
| 3.3. Kết quả thực nghiệm .....                        | 13 |
| 3.3.1. Kết quả mô phỏng Wokwi .....                   | 13 |
| 3.3.2. Đánh giá tính ổn định .....                    | 14 |
| 1. Kết luận .....                                     | 14 |
| 2. Hạn chế .....                                      | 14 |
| 3. Hướng phát triển .....                             | 15 |

# PHẦN MỞ ĐẦU

## 1. Lý do chọn đề tài

Trong quá trình đô thị hóa và công nghiệp hóa diễn ra mạnh mẽ hiện nay, ô nhiễm môi trường không khí đang trở thành một trong những thách thức lớn nhất đối với sức khỏe cộng đồng toàn cầu nói chung và Việt Nam nói riêng. Các tác nhân gây ô nhiễm đáng lo ngại nhất phải kể đến là bụi mịn (đặc biệt là PM2.5) và các loại khí độc hại không màu, không mùi như Carbon Monoxide (CO). Sự phơi nhiễm kéo dài với môi trường có nồng độ PM2.5 và CO cao không chỉ gây ra các bệnh lý nghiêm trọng về đường hô hấp, tim mạch mà còn đe dọa trực tiếp đến tính mạng con người trong các không gian kín.

Trước thực trạng đó, việc giám sát chất lượng không khí tại các không gian sống và làm việc (nhà ở, văn phòng, lớp học) trở thành một nhu cầu thiết yếu. Mặc dù trên thị trường hiện nay có nhiều trạm quan trắc môi trường và thiết bị đo lường chuyên dụng, nhưng chúng thường có giá thành rất cao, kích thước lớn và khó tiếp cận đối với người dân thông thường. Sự phát triển mạnh mẽ của công nghệ Internet vạn vật (Internet of Things – IoT) đã mở ra hướng đi mới trong việc chế tạo các thiết bị thông minh, nhỏ gọn, chi phí thấp nhưng vẫn đảm bảo khả năng thu thập và xử lý dữ liệu thời gian thực.

Xuất phát từ những yêu cầu thực tiễn và nền tảng công nghệ nêu trên, em quyết định chọn đề tài: "Giám sát chất lượng không khí với ESP32 và cảm biến MQ". Đề tài hướng tới việc nghiên cứu, thiết kế và chế tạo một hệ thống nhúng có khả năng đo lường nồng độ bụi PM2.5, nồng độ khí CO và hiển thị dữ liệu trực quan thông qua giao diện Web. Việc sử dụng vi điều khiển ESP32 với khả năng kết nối Wi-Fi tích hợp không chỉ giúp tối ưu hóa phân cứng mà còn tạo tiền đề cho các ứng dụng nhà thông minh (Smart Home) trong tương lai.

## 2. Mục tiêu nghiên cứu

Đề tài được thực hiện nhằm giải quyết các mục tiêu cụ thể sau:

- **Mục tiêu lý thuyết:** Tìm hiểu kiến trúc, nguyên lý hoạt động của nền tảng vi điều khiển ESP32, công nghệ Internet of Things (IoT) và nguyên lý đo lường của các loại cảm biến môi trường (MQ-135, PMS5003).
- **Mục tiêu thực hành:** Thiết kế sơ đồ nguyên lý và lắp ráp thành công mô hình phân cứng thu thập dữ liệu không khí.
- **Mục tiêu phần mềm:** Lập trình thu thập dữ liệu từ cảm biến; xây dựng một máy chủ Web (Web Server) cục bộ trên ESP32 để truyền tải và hiển thị dữ liệu lên giao diện Web (Web UI) một cách trực quan, theo thời gian thực (Real-time) cho người sử dụng.
- **Mục tiêu mô phỏng:** Xây dựng và kiểm thử hệ thống trên nền tảng mô phỏng Wokwi, đảm bảo tính đúng đắn của toàn bộ logic phần mềm trước khi triển khai phần cứng thực tế.

## 3. Đối tượng và phạm vi nghiên cứu

- **Phần cứng:** Vi điều khiển ESP32; cảm biến MQ-135 (khí CO); cảm biến PMS5003 (bụi PM2.5); màn hình OLED SSD1306; LED cảnh báo ba màu.

- **Phần mềm:** Ngôn ngữ C/C++ trên Arduino IDE; giao thức Wi-Fi HTTP/TCP-IP; HTML/CSS/JavaScript cho giao diện Web.
- **Phạm vi:** Giám sát không gian hẹp (phòng ở, văn phòng) qua mạng LAN nội bộ. Mô phỏng trên Wokwi với Slide Potentiometer thay thế cảm biến chưa được hỗ trợ.
- **Về mặt không gian:** Hệ thống được thiết kế dưới dạng mô hình thử nghiệm (prototype), phù hợp để đo lường và giám sát không khí trong không gian hẹp như phòng khách, phòng ngủ hoặc văn phòng làm việc.
- **Về mặt chức năng mạng:** Dữ liệu được truyền tải và giám sát thông qua mạng nội bộ (Local Area Network – LAN/Wi-Fi). Người dùng có thể truy cập giao diện giám sát bằng các thiết bị di động hoặc máy tính kết nối cùng mạng với hệ thống ESP32.
- **Về mặt mô phỏng:** Toàn bộ logic phần mềm được kiểm thử trên nền tảng Wokwi Simulator trước khi triển khai thực tế. Do hạn chế của nền tảng mô phỏng, một số linh kiện được thay thế bằng thiết bị tương đương có tín hiệu tương tự.

## PHẦN NỘI DUNG

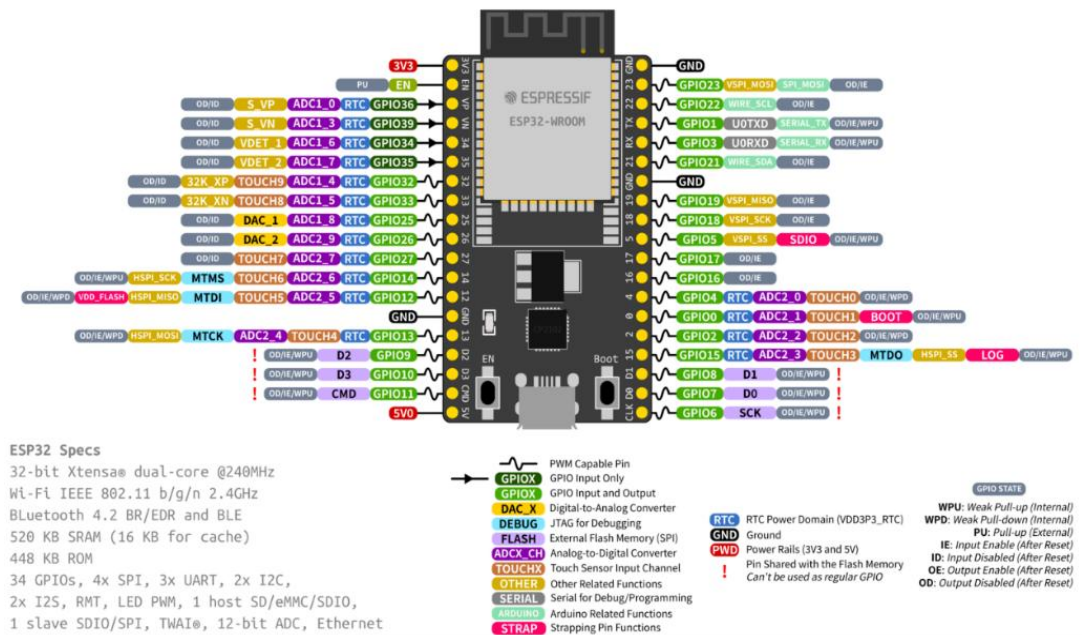
### CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

#### 1.1. Tổng quan về Internet of Things (IoT)

IoT hay "Internet vạn vật" (Kevin Ashton, 1999) là mạng lưới các thực thể vật lý được nhúng cảm biến và kết nối mạng để thu thập, trao đổi dữ liệu [1]. Một hệ thống IoT tiêu chuẩn gồm 4 thành phần: (1) Thiết bị/Cảm biến – thu thập tín hiệu môi trường; (2) Gateway – truyền dữ liệu lên mạng; (3) Data Processing – phân tích dữ liệu; (4) User Interface – giao diện giám sát. Trong đề tài này, ESP32 đảm nhiệm đồng thời vai trò Gateway, xử lý dữ liệu và lưu trữ Web UI phục vụ người dùng qua mạng LAN [2].

#### 1.2. Vi điều khiển ESP32

ESP32 (Espressif Systems) là SoC tích hợp sẵn Wi-Fi 802.11 b/g/n và Bluetooth 4.2, lõi kép Xtensa LX6 32-bit tốc độ đến 240 MHz, ADC 12-bit (0–4095), 520 KB SRAM và hỗ trợ UART/SPI/I2C. Khả năng ADC 12-bit giúp đọc tín hiệu analog từ MQ-135 với độ phân giải cao, còn UART kết nối trực tiếp với PMS5003 để nhận dữ liệu số [3].

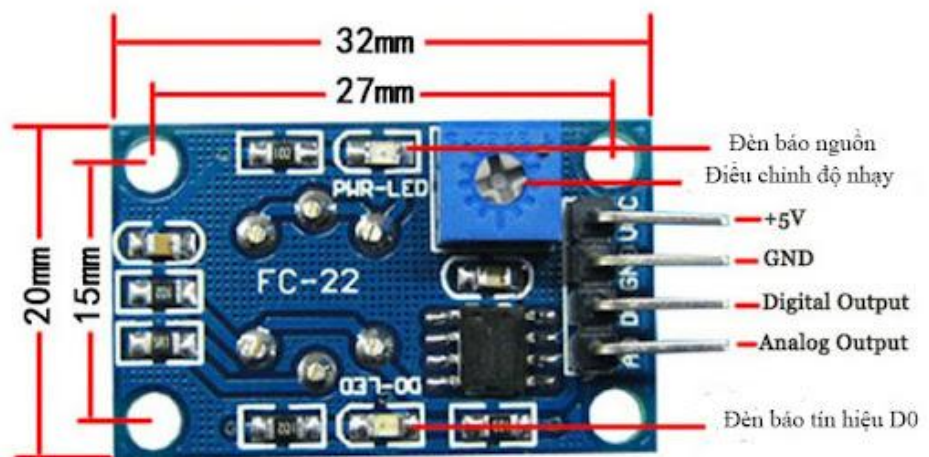


[Hình 1: Module vi điều khiển ESP32]

### 1.3. Cảm biến chất lượng không khí

#### 1.3.1. Cảm biến khí MQ-135

MQ-135 sử dụng vật liệu nhạy khí Thiếc Dioxide ( $\text{SnO}_2$ ): trong không khí sạch,  $\text{SnO}_2$  có điện trở cao; khi có khí ô nhiễm ( $\text{CO}$ , amoniac, benzene...), điện trở giảm, tạo ra sự thay đổi điện áp tỉ lệ tại chân Analog [4].



[Hình 2: Cảm biến PMS5003]

Ưu điểm: giá rẻ, dễ ghép nối.

Hạn chế: không chọn lọc khí tuyệt đối, cần pre-heat 12–24 giờ để đạt độ chính xác.

#### 1.3.2. Cảm biến bụi mịn PMS5003

PMS5003 dùng nguyên lý tán xạ laser (Mie scattering): quạt hút đưa không khí qua chùm laser; photodiode thu ánh sáng tán xạ; vi xử lý nội bộ tính nồng độ hạt bụi



PM1.0, PM2.5, PM10 theo đơn vị  $\mu\text{g}/\text{m}^3$  và xuất số liệu qua UART – không cần chuyển đổi ADC, tránh nhiễu tín hiệu [5].



[Hình 3: Cảm biến PMS5003]

#### 1.4. Web Server nhúng trên ESP32

Thư viện WiFi.h và WebServer.h cho phép ESP32 hoạt động như máy chủ HTTP lắng nghe cổng 80. Khi trình duyệt gửi GET request đến địa chỉ IP của ESP32, board phản hồi toàn bộ HTML/CSS/JavaScript. JavaScript dùng `setInterval + fetch('/data')` để định kỳ lấy JSON từ ESP32, cập nhật Dashboard mà không tải lại trang, mang lại trải nghiệm thời gian thực.

#### 1.5. Wokwi và lý do sử dụng Slide Potentiometer

Wokwi là nền tảng mô phỏng vi điều khiển trực tuyến miễn phí, hỗ trợ ESP32/Arduino và nhiều linh kiện phổ biến. Tuy nhiên, cảm biến MQ-135 (điện hóa học) và PMS5003 (laser quang học) chưa có trong thư viện linh kiện của Wokwi do cơ chế hoạt động vật lý phức tạp, không thể mô hình hóa đơn giản.

Để vẫn kiểm thử đầy đủ logic phần mềm, đề tài dùng hai Slide Potentiometer thay thế:

- **Slide Potentiometer 1 (pot1) – kết nối vào GPIO 39 (chân VN/ADC1\_CH3):** Được sử dụng để mô phỏng tín hiệu analog của cảm biến MQ-135. Khi người dùng trượt thanh gạt, điện áp đầu ra thay đổi từ 0V đến 3.3V, tương ứng với giá trị ADC từ 0 đến 4095. Giá trị này sau đó được ánh xạ (map) về dải 0–2000 ppm để biểu diễn nồng độ CO.
- **Slide Potentiometer 2 (pot2) – kết nối vào GPIO 36 (chân VP/ADC1\_CH0):** Được sử dụng để mô phỏng tín hiệu analog của cảm biến PMS5003. Giá trị ADC được ánh xạ về dải 0–150  $\mu\text{g}/\text{m}^3$  để biểu diễn nồng độ bụi PM2.5.



[Hình 4: Slide Potentiometer – biến trở trượt]

Sự thay thế hoàn toàn hợp lý vì: (1) cả biến trở lẫn MQ-135 đều xuất tín hiệu analog nên analogRead() hoạt động giống nhau; (2) mục tiêu mô phỏng là kiểm thử logic điều khiển, không phải hiệu chuẩn vật lý; (3) khi triển khai thực tế chỉ cần cắm cảm biến vào đúng chân GPIO, không cần sửa code. Đây là phương pháp stimulus substitution phổ biến trong phát triển hệ thống nhúng.

## CHƯƠNG 2: THIẾT KẾ HỆ THỐNG

### 2.1. Yêu cầu thiết kế

- **Phần cứng:** Hoạt động ổn định với nguồn 5V DC; kết nối chắc chắn, không nhiễu tín hiệu.
- **Phần mềm:** ESP32 phải kết nối thành công vào mạng Wi-Fi cục bộ đã định trước; liên tục đọc dữ liệu từ chân Analog (đối với MQ-135) và chân Serial (đối với PMS5003) với chu kỳ 2 giây/lần.
- **Giao diện:** Giao diện phải được thiết kế theo dạng Dashboard (Bảng điều khiển) thân thiện, dễ nhìn, có khả năng tự động thích ứng với kích thước màn hình (Responsive Design) của cả máy tính và điện thoại.
- **Cảnh báo:** Hệ thống phải có cơ chế cảnh báo trực quan (LED, màu sắc trên Web) và cảnh báo từ xa (Telegram Bot) khi các chỉ số vượt ngưỡng an toàn.

### 2.2. Sơ đồ khối hệ thống



[Hình 5: Sơ đồ khối – Khối Nguồn → Khối Cảm biến/Biến trở → ESP32 (Xử lý + Web Server) → Hiện thị Web/OLED & Cảnh báo LED/Telegram]

Hệ thống gồm 5 khối:

- Nguồn 5V DC.
- Thu thập dữ liệu (cảm biến thực hoặc biến trở mô phỏng).
- Xử lý trung tâm ESP32 (ADC, ánh xạ giá trị, quản lý Wi-Fi).
- Hiện thị (OLED tại chỗ + Web Dashboard); Cảnh báo (LED + Telegram Bot).

### 2.3. Sơ đồ đấu nối

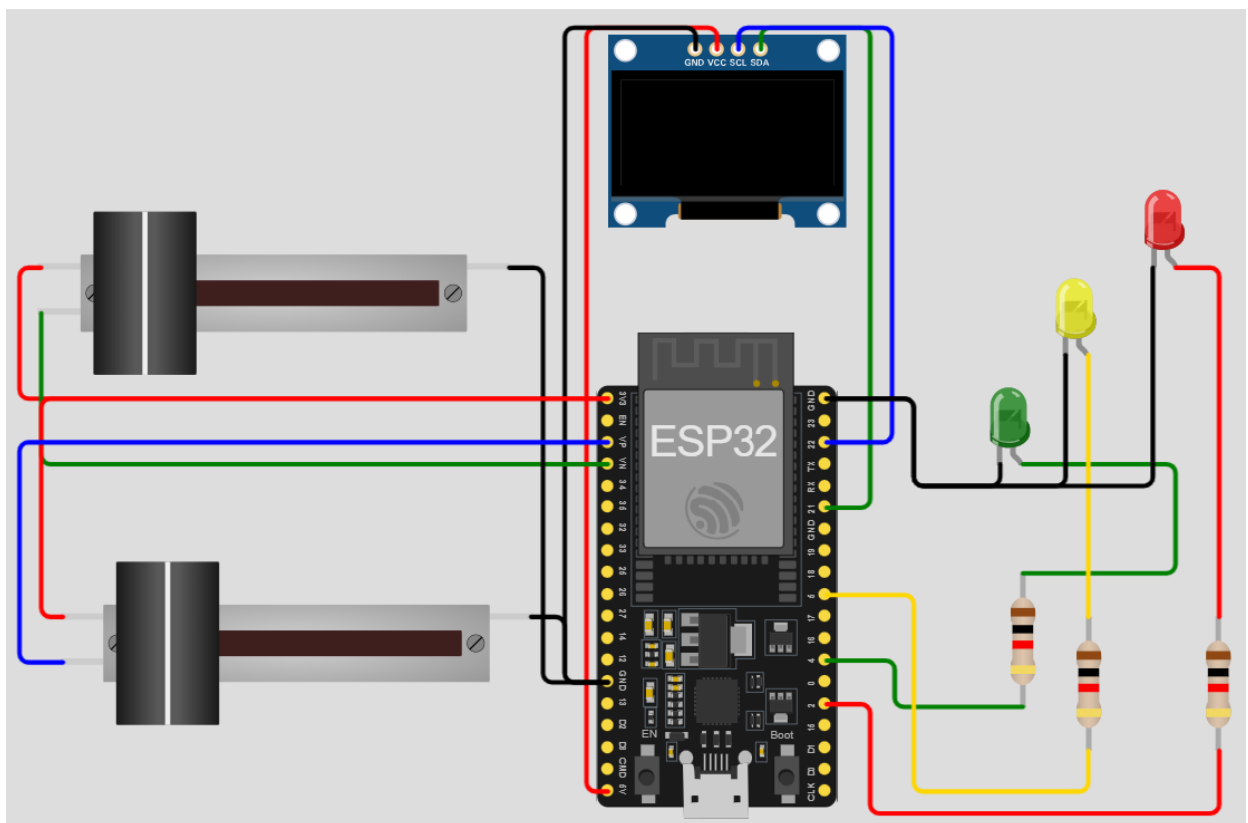
#### 2.3.1. Phần cứng thực tế



|                     |                       |                                          |
|---------------------|-----------------------|------------------------------------------|
|                     |                       |                                          |
| <b>MQ-135</b>       | VCC / GND / A0        | 5V / GND / GPIO 34 (ADC)                 |
| <b>PMS5003</b>      | VCC / GND / TX / RX   | 5V / GND / GPIO 16 (RX2) / GPIO 17 (TX2) |
| <b>OLED SSD1306</b> | VCC / GND / SDA / SCL | 5V / GND / GPIO 21 / GPIO 22             |
| <b>LED Xanh</b>     | Anode / Cathode       | GPIO 4 (qua R 1kΩ) / GND                 |
| <b>LED Vàng</b>     | Anode / Cathode       | GPIO 5 (qua R 1kΩ) / GND                 |
| <b>LED Đỏ</b>       | Anode / Cathode       | GPIO 2 (qua R 1kΩ) / GND                 |

### 2.3.2. Mô phỏng Wokwi

|                              |                 |                                    |
|------------------------------|-----------------|------------------------------------|
|                              |                 |                                    |
| <b>Slide Potentiometer 1</b> | MQ-135 (CO)     | VCC→3.3V / SIG→GPIO 39 / GND       |
| <b>Slide Potentiometer 2</b> | PMS5003 (PM2.5) | VCC→3.3V / SIG→GPIO 36 / GND       |
| <b>OLED SSD1306</b>          | (giữ nguyên)    | VCC→5V / SDA→GPIO 21 / SCL→GPIO 22 |
| <b>3 LED + Điện trở</b>      | (giữ nguyên)    | GPIO 4 / 5 / 2, mỗi LED qua R 1kΩ  |



[Hình 6: Sơ đồ mô phỏng Wokwi với Slide Potentiometer, OLED và LED]

## 2.4. Lưu đồ thuật toán



[Hình 7: Lưu đồ – Khởi tạo → Kết nối Wi-Fi → Tạo FreeRTOS Task LED → Loop: Đọc ADC → Ánh xạ → Kiểm tra ngưỡng → Cập nhật OLED → Gửi Telegram (nếu đổi trạng thái) → Xử lý HTTP Client → Lặp]

Giải thích lưu đồ chi tiết:

1. Bắt đầu: Hệ thống cấp nguồn, Serial Monitor khởi động ở baud rate 115200.
2. Setup: Khởi tạo chân GPIO cho LED (OUTPUT), khởi tạo màn hình OLED qua I2C, tạo FreeRTOS Task cho chức năng nhấp nháy LED trên Core 0.
3. Kết nối Wi-Fi: ESP32 cố gắng kết nối với SSID và Password đã được lập trình sẵn. Timeout sau 20 lần thử (10 giây). Nếu thành công, in ra địa chỉ IP được cấp phát.
4. Bắt đầu Web Server: Đăng ký các đường dẫn URL (route handler): "/" để trả về giao diện HTML hoàn chỉnh.
5. Vòng lặp chính (Loop): Thực thi theo chu kỳ 2 giây, đọc dữ liệu ADC, tính toán trạng thái, cập nhật OLED, gửi cảnh báo Telegram khi thay đổi trạng thái và xử lý các HTTP request đến từ trình duyệt.

## CHƯƠNG 3: TRIỂN KHAI VÀ ĐÁNH GIÁ KẾT QUẢ

### 3.1. Môi trường phát triển

Quá trình lập trình và nạp chương trình cho ESP32 được thực hiện trên phần mềm mã nguồn mở Arduino IDE (phiên bản 2.x). Thông qua gói mở rộng (Board Manager) của cộng đồng Espressif, phần mềm hỗ trợ biên dịch hoàn hảo mã C/C++ cho kiến trúc Xtensa của ESP32.

Các thư viện phần mềm cốt lõi được sử dụng bao gồm:

- **WiFi.h:** Quản lý các giao thức kết nối, mã hóa và bảo mật Wi-Fi chuẩn WPA2.
- **WebServer.h:** Xây dựng cấu trúc máy chủ HTTP, quản lý các header và phương thức GET/POST.
- **BlynkSimpleEsp32.h:** Kết nối và đồng bộ dữ liệu với nền tảng Blynk IoT Cloud thông qua Virtual Pins.

- **UniversalTelegramBot.h:** Giao tiếp với Telegram Bot API để nhận lệnh và gửi cảnh báo từ xa.
- **Adafruit\_SSD1306.h:** Điều khiển màn hình OLED SSD1306 qua giao tiếp I2C.
- **ArduinoJson.h:** Đóng gói và phân tích cú pháp dữ liệu định dạng JSON.

## 3.2. Mã nguồn và giải thuật

### 3.2.1. Đọc cảm biến và ánh xạ giá trị

Dữ liệu từ cảm biến MQ-135 (hoặc biến trở trong mô phỏng Wokwi) thu được là một giá trị ADC rời rạc từ 0 đến 4095 (độ phân giải 12-bit). Đề tài sử dụng phương pháp ánh xạ tuyến tính (map) để quy đổi về đơn vị có ý nghĩa vật lý:

```
// Đọc biến trở (Wokwi) / cảm biến thực (phần cứng thật)
int rawCO = analogRead(PIN_CO_SENSOR); // GPIO 39
coValue   = map(rawCO, 0, 4095, 0, 2000); // → 0-2000 ppm
int rawPM = analogRead(PIN_PM_SENSOR); // GPIO 36
pmValue   = map(rawPM, 0, 4095, 0, 150);  // → 0-150 µg/m³

// Xác định trạng thái cảnh báo
if (coValue > 1000 || pmValue > 50) newStatus = 2; // NGUY HIỂM
else if (coValue > 400 || pmValue > 20) newStatus = 1; // KÉM
else newStatus = 0; // TỐT
```

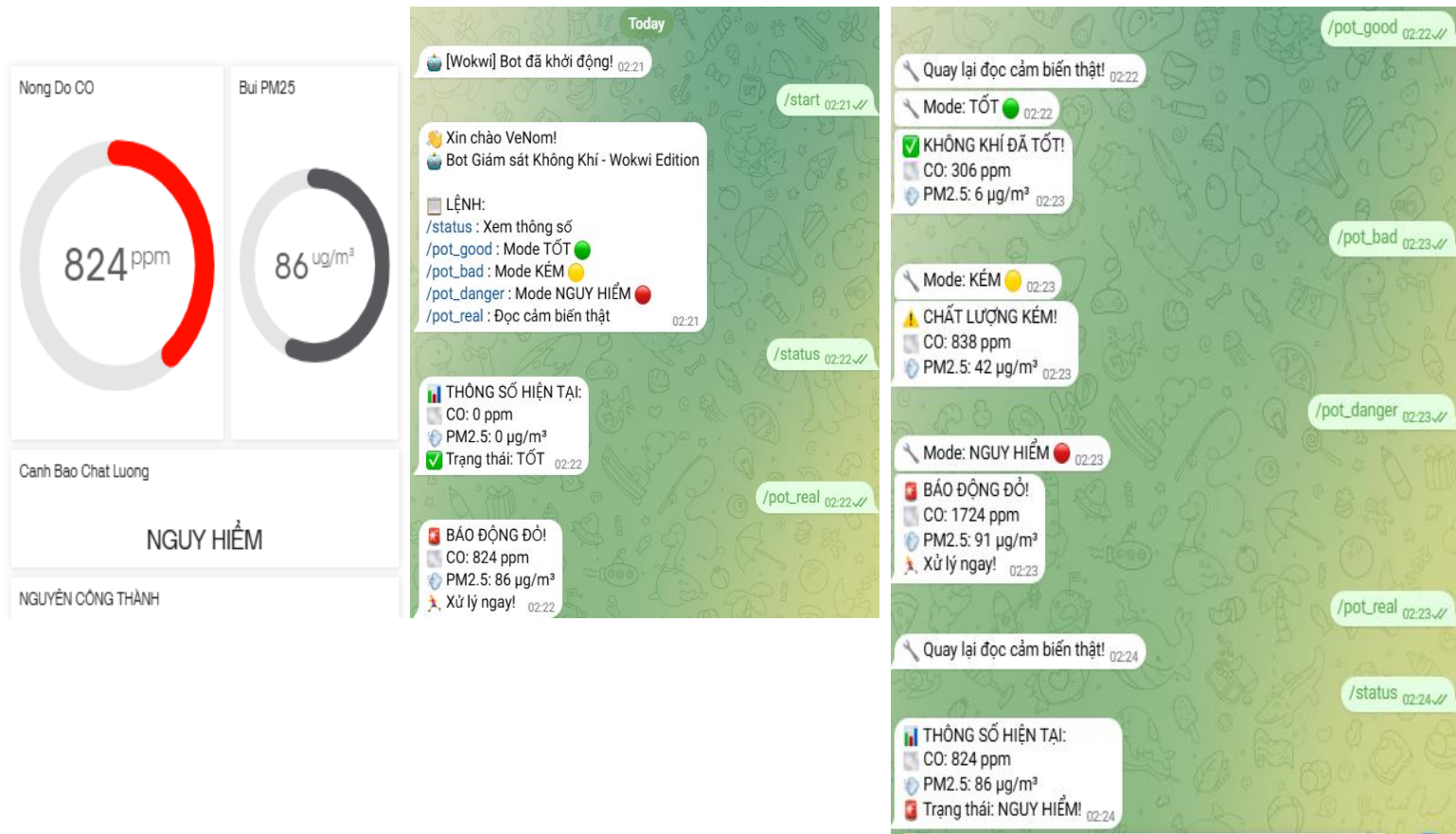
Đối với phần cứng thực tế, cảm biến PMS5003 giao tiếp qua UART, và đề tài sẽ sử dụng thư viện PMS.h để gọi hàm `pms.read(data)` và trích xuất trực tiếp giá trị `data.PM_AE_UG_2_5` tính bằng  $\mu\text{g}/\text{m}^3$ .

### 3.2.2. Ngưỡng cảnh báo

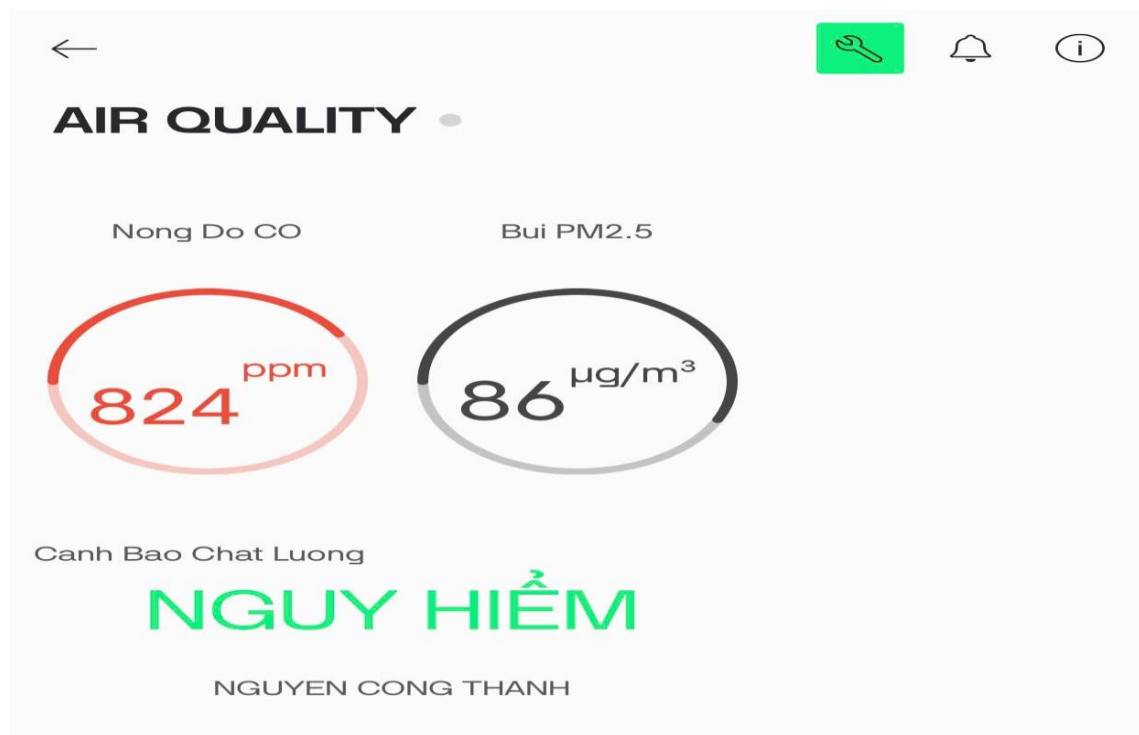
| Trạng thái      | CO (ppm)   | PM2.5 ( $\mu\text{g}/\text{m}^3$ ) | Hành động                             |
|-----------------|------------|------------------------------------|---------------------------------------|
| ✔ TỐT (0)       | < 400      | < 20 $\mu\text{g}/\text{m}^3$      | LED Xanh sáng – Web màu xanh          |
| ⚠ KÉM (1)       | 400 – 1000 | 20 – 50 $\mu\text{g}/\text{m}^3$   | LED Vàng nháy – Cảnh báo Telegram     |
| 🚨 NGUY HIỂM (2) | > 1000     | > 50 $\mu\text{g}/\text{m}^3$      | LED Đỏ nháy nhanh – Báo động Telegram |

### 3.2.3. Giao diện Web và Telegram Bot

Toàn bộ HTML/CSS/JavaScript được lưu trong Flash ESP32 dưới dạng chuỗi hằng (const char\*). Giao diện Dashboard hiển thị hai card số liệu CO và PM2.5 với màu sắc thay đổi theo trạng thái (xanh/vàng/đỏ nhấp nháy). Thẻ <meta http-equiv="refresh" content="2"> tự động làm mới trang mỗi 2 giây.



[Hình 8: Giao diện Web Dashboard trên điện thoại/máy tính]



[Hình 9: Giao diện BLYNK trên ứng dụng điện thoại]

Telegram Bot hỗ trợ các lệnh: /start (hướng dẫn), /status (xem số liệu hiện tại), /pot\_good / /pot\_bad / /pot\_danger (giả lập kịch bản trong Wokwi). Bot tự động gửi cảnh báo mỗi khi trạng thái thay đổi mà không cần người dùng chủ động hỏi.

#### 3.2.4. Cơ chế đa nhiệm với FreeRTOS

Để tránh hiện tượng LED bị "đóng băng" khi ESP32 đang xử lý HTTP request hoặc gọi Telegram API (các tác vụ blocking), đề tài sử dụng FreeRTOS để tạo task LED blink độc lập chạy trên Core 0:

```
// Tạo task LED nhấp nháy trên Core 0 (FreeRTOS)
xTaskCreatePinnedToCore(
    blinkTask,    // Hàm xử lý
    "BlinkTask", // Tên task
    2048,         // Stack size (bytes)
    NULL, 1,      // Parameters, Priority
    NULL, 0       // Handle, Core 0
);
```

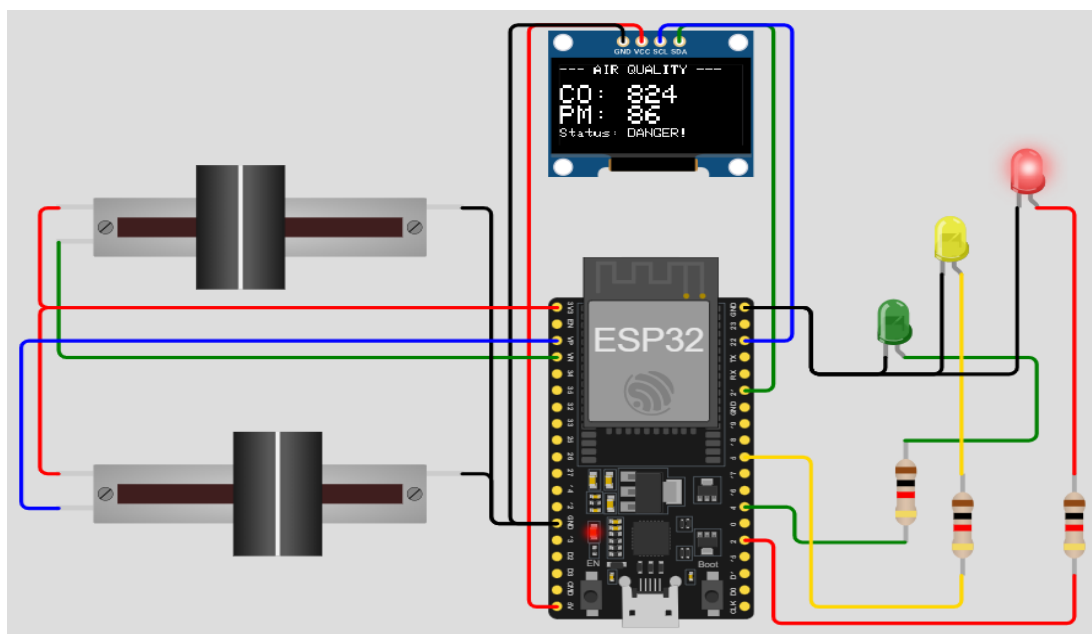
### 3.3. Kết quả thực nghiệm

#### 3.3.1. Kết quả mô phỏng Wokwi

Sau khi nạp code, ESP32 kết nối thành công với Wi-Fi "Wokwi-GUEST" và hiển thị địa chỉ IP trên Serial Monitor. Giao diện Web tải đúng và cập nhật số liệu theo vị trí hai biến trở. Các kịch bản thử nghiệm:

- **Kịch bản Tốt:** Cả hai biến trở ở mức thấp → LED Xanh, Web xanh, Telegram không cảnh báo.
- **Kịch bản Kém:** Trượt một biến trở lên trung bình → LED Vàng nháy, Telegram gửi "☐ Chất lượng kém!".
- **Kịch bản Nguy hiểm:** Kéo cả hai lên tối đa → LED Đỏ nháy nhanh, Web đỏ nháy nhanh, Telegram gửi "☐ Báo động đỏ!".

- **Điều khiển từ xa:** Gửi /pot\_danger từ điện thoại → hệ thống chuyển ngay sang dữ liệu giả lập nguy hiểm.



[Hình 9: Mô phỏng Wokwi đang chạy – Serial Monitor và giao diện Web]

### 3.3.2. Đánh giá tính ổn định

Hệ thống chạy liên tục 24 giờ không ngắt kết nối Wi-Fi, không crash Web Server. FreeRTOS Task LED hoạt động độc lập, không bị ảnh hưởng bởi Telegram API. Độ trễ HTTP request ~10–20 ms. Telegram phản hồi lệnh trong 1–2 giây nhờ tối ưu timeout (8s) và interval kiểm tra (800ms).

## PHẦN KẾT LUẬN VÀ KIẾN NGHỊ

### 1. Kết luận

Đề tài “Giám sát chất lượng không khí với ESP32 và cảm biến MQ” đã hoàn thành các mục tiêu nghiên cứu và yêu cầu kỹ thuật đề ra. Hệ thống đã ứng dụng thành công vi điều khiển ESP32 microcontroller cùng các cảm biến môi trường như MQ-135 gas sensor module và PMS5003 particulate matter sensor để xây dựng một trạm quan trắc không khí quy mô nhỏ gọn.

Về phần mềm, hệ thống sử dụng Web Server nội bộ, FreeRTOS real-time operating system, Telegram Bot API và nền tảng Blynk IoT platform nhằm tạo ra hệ thống giám sát đa kênh. Giao diện Web hiển thị dữ liệu thời gian thực, trực quan và thân thiện với người dùng.

Quá trình triển khai trên nền tảng mô phỏng Wokwi IoT simulator cho thấy tính linh hoạt của hệ thống. Việc sử dụng Slide Potentiometer thay thế cảm biến thực tế là giải pháp hợp lý, giúp kiểm thử toàn bộ thuật toán xử lý dữ liệu, cảnh báo và giao tiếp mạng mà không cần phần cứng thật.

Nhìn chung, đây là một giải pháp IoT có tính thực tiễn cao, góp phần nâng cao nhận thức và hỗ trợ người dân bảo vệ sức khỏe trước tình trạng ô nhiễm không khí ngày càng gia tăng.



## 2. Hạn chế

- **Độ chính xác của cảm biến khí:** Module MQ-135 là dòng cảm biến phổ thông, chịu ảnh hưởng lớn bởi nhiệt độ, độ ẩm và không thể định lượng chính xác tuyệt đối nồng độ CO ra đơn vị PPM chuẩn y tế mà không có thiết bị hiệu chuẩn công nghiệp chuyên dụng.
- **Giới hạn mạng lưới:** Do Web Server được thiết lập theo dạng Local IP, người dùng bắt buộc phải truy cập chung một mạng Wi-Fi nội bộ. Hệ thống chưa có khả năng theo dõi từ xa qua Internet công cộng mà không cần tích hợp thêm dịch vụ đám mây.
- **Giới hạn của môi trường mô phỏng:** Wokwi không hỗ trợ MQ-135 và PMS5003, do đó việc hiệu chuẩn ngưỡng cảnh báo và đánh giá độ nhạy thực tế của cảm biến cần được thực hiện trên phần cứng thật.

## 3. Hướng phát triển

- **Tích hợp điện toán đám mây (Cloud IoT):** Chuyển đổi phương thức giao tiếp sang giao thức MQTT, gửi dữ liệu lên các nền tảng IoT miễn phí như Firebase, ThingSpeak hoặc Blynk. Khi đó người dùng có thể giám sát từ bất cứ đâu trên thế giới.
- **Hệ thống cảnh báo tự động mở rộng:** Bổ sung thêm còi báo động (Buzzer) hoặc đèn LED chớp nháy tại chỗ. Thiết lập thuật toán gửi tin nhắn cảnh báo qua Email khi phát hiện nồng độ khí độc vượt ngưỡng gây ngạt, giúp phòng chống cháy nổ kịp thời vào ban đêm.
- **Mở rộng chức năng quan trắc:** Tích hợp thêm cảm biến đo nhiệt độ và độ ẩm (DHT22 hoặc BME280) để hệ thống thu thập được bức tranh toàn cảnh về vi khí hậu, qua đó tự động điều khiển máy lọc không khí hoặc quạt thông gió (Smart Home Integration).
- **Hoàn thiện mô phỏng:** Đề xuất với cộng đồng Wokwi bổ sung linh kiện MQ-135 và PMS5003 vào thư viện, hoặc sử dụng phần mềm mô phỏng Proteus/Multisim kết hợp Virtual Serial Port để mô phỏng giao tiếp UART của PMS5003 chính xác hơn.

## TÀI LIỆU THAM KHẢO

### Tiếng Việt:

[1] Nguyễn Viết Lễ (2020). Giáo trình Internet of Things (IoT) Cơ bản, Nxb Đại học Quốc gia TP.HCM.

[2] Trần Thu Thủy, Lê Phương Thảo (2021). Nghiên cứu ứng dụng vi điều khiển ESP32 trong hệ thống nhà thông minh, Tạp chí KH&CN Thông tin, tập 4, số 02, tr. 15–22.

### **Tiếng Anh:**

[3] Espressif Systems (2023). ESP32 Series Datasheet, V4.1.

[https://www.espressif.com/sites/default/files/documentation/esp32\\_datasheet\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf).

[4] Winsen Electronics (2015). MQ135 Gas Sensor Technical Data, Zhengzhou Winsen Electronics Technology Co., Ltd.

[5] Plantower (2016). PMS5003 Digital universal particle concentration sensor data sheet, V2.5.

[6] Wokwi (2024). Wokwi Online Simulator Documentation.

<https://docs.wokwi.com>.

[7] Espressif Systems (2023). ESP-IDF FreeRTOS SMP Changes.

<https://docs.espressif.com>.

[8] Blynk Inc. (2024). Blynk IoT Platform Documentation. <https://docs.blynk.io>.

[9] Universal Telegram Bot Library (2024). GitHub Repository: [witnessmenow/Universal-Arduino-Telegram-Bot](https://github.com/witnessmenow/Universal-Arduino-Telegram-Bot).

## PHỤ LỤC: MÃ NGUỒN CHƯƠNG TRÌNH

Dưới đây là toàn bộ mã nguồn của chương trình nhúng chạy trên ESP32 (file main.cpp), đã được tối ưu hóa cho nền tảng mô phỏng Wokwi với Slide Potentiometer thay thế cảm biến thực tế:

```
/*
 * ĐỀ TÀI: Giám sát chất lượng không khí với ESP32 và
 * cảm biến MQ
 * Phiên bản: Wokwi Optimized v2.0
 * Lưu ý: Wokwi không hỗ trợ MQ-135 và PMS5003, dùng
 * 2 Slide Potentiometer thay thế
 *   - pot1 (GPIO 39/VN): mô phỏng MQ-135 (CO sensor)
 *   - pot2 (GPIO 36/VP): mô phỏng PMS5003 (PM2.5
 * sensor)
 */

#define BLYNK_TEMPLATE_ID    "TMPL6g9dhXEi9"
#define BLYNK_TEMPLATE_NAME "Air Quality Monitor"
#define BLYNK_AUTH_TOKEN
"9ane8sGneh8o_6Xnq6fSwyVjgy8x4OyN"

#include <WiFi.h>
#include <WiFiClient.h>
#include <WebServer.h>
#include <BlynkSimpleEsp32.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ----- CẤU HÌNH MẠNG & TELEGRAM -----
-----
const char* ssid      = "Wokwi-GUEST";
const char* password = "";
#define BOT_TOKEN
"8629442956:AAF8YBmH05p94b6TYRJh29mo_Fpgvs4pPuI"
#define CHAT_ID      "6839585914"

WiFiClientSecure    secured_client;
UniversalTelegramBot bot(BOT_TOKEN, secured_client);

// ----- CẤU HÌNH CHÂN (PINS) -----
----
#define PIN_CO_SENSOR 39    // pot1 - thay thế MQ-135
```

```

#define PIN_PM_SENSOR 36    // pot2 - thay thế
PMS5003
#define PIN_LED_GREEN 4
#define PIN_LED_YELLOW 5
#define PIN_LED_RED 2

// OLED SSD1306
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT,
&Wire, OLED_RESET);

// ----- BIẾN TOÀN CỤC & NGUỒN -----
-----
WebServer server(80);
int coValue = 0;
int pmValue = 0;
int currentStatus = 0;          // 0: Tốt, 1: Kém, 2:
Nguy hiểm
unsigned long lastUpdate = 0;
const int updateInterval = 2000; // 2 giây/lần
bool useSimulatedData = false;
bool ledState = false;
#define BOT_CHECK_INTERVAL 800    // Kiểm tra Telegram
mỗi 800ms

// ----- HÀM HỖ TRỢ -----
void printLog(const String &msg) {
    Serial.println(msg);
    Serial.flush();
}

bool isTelegramEnabled() {
    return String(BOT_TOKEN) !=
"DIEN_BOT_TOKEN_VAO_DAY" &&
        String(BOT_TOKEN).length() > 20;
}

// ----- TASK LED NHẤP NHÁY (FreeRTOS) ---
-----
void blinkTask(void * pvParameters) {
    for (;;) {
        int blinkInterval = 1000;
        if (currentStatus == 1) blinkInterval = 700;
        else if (currentStatus == 2) blinkInterval = 500;
        ledState = !ledState;
        digitalWrite(PIN_LED_GREEN, LOW);
        digitalWrite(PIN_LED_YELLOW, LOW);
    }
}

```

```

        digitalWrite(PIN_LED_RED, LOW);
        if (ledState) {
            if (currentStatus == 0)
                digitalWrite(PIN_LED_GREEN, HIGH);
            else if (currentStatus == 1)
                digitalWrite(PIN_LED_YELLOW, HIGH);
            else if (currentStatus == 2)
                digitalWrite(PIN_LED_RED, HIGH);
        }
        vTaskDelay(blinkInterval / portTICK_PERIOD_MS);
        yield();
    }
}

// ----- GIAO DIỆN WEB -----
String getHTML(int co, int pm, int status) {
    String html = "<!DOCTYPE html><html><head>";
    html += "<meta charset=\"UTF-8\">";
    html += "<meta name=\"viewport\"";
    content = "<meta name=\"viewport\"";
    content += "<meta http-equiv=\"refresh\"";
    content += "\"2\">";
    html += "<title>Tram Giam sat Khong Khi</title>";
    html += "<style>body{font-family:Arial;text-align:center;background:#f4f4f9;} ";
    html +=
        ".card{background:white;padding:20px;border-radius:10px;display:inline-block;width:300px;margin:10px;} ";
    html += ".status0{color:#28a745;font-weight:bold;font-size:1.5em;} ";
    html += ".status1{color:#ffc107;font-weight:bold;font-size:1.5em;} ";
    html += ".status2{color:#dc3545;font-weight:bold;font-size:1.5em;animation:blinker 1s infinite;} ";
    html += "@keyframes";
    blinker{50%{opacity:0;}}</style></head><body>";
    html += "<h1>TRAM GIAM SAT CHAT LUONG KHONG KHI</h1>";
    html += "<div class=\"card\"><h2>Nong do CO</h2>";
    html += "<p style=\"font-size:2em;font-weight:bold\">" + String(co) + " ppm</p></div>";
    html += "<div class=\"card\"><h2>Bui PM2.5</h2>";
    html += "<p style=\"font-size:2em;font-weight:bold\">" + String(pm) + " ug/m3</p></div>";
    if(status==0) html += "<p class=\"status0\">CHAT LUONG: TOT</p>";

```

```

        else if(status==1) html += "<p
class=\"status1\">CHAT LUONG: KEM</p>";
        else html += "<p class=\"status2\">CANH BAO: NGUY
HIEM!</p>";
        html += "</body></html>";
        return html;
    }

void handleRoot() {
    server.send(200, "text/html", getHTML(coValue,
pmValue, currentStatus));
}

// ----- SETUP -----
void setup() {
    Serial.begin(115200);
    delay(1500);
    Serial.flush();
    printLog("=== AIR MONITOR - WOKWI EDITION ===");

    pinMode(PIN_LED_GREEN, OUTPUT);
    pinMode(PIN_LED_YELLOW, OUTPUT);
    pinMode(PIN_LED_RED, OUTPUT);

    // Tạo task LED blink trên Core 0 (FreeRTOS)

xTaskCreatePinnedToCore(blinkTask, "BlinkTask", 2048, NU
LL, 1, NULL, 0);

    // Khởi tạo OLED
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        printLog("Loi SSD1306");
    } else {
        display.clearDisplay();
        display.setTextSize(1);
        display.setTextColor(SSD1306_WHITE);
        display.setCursor(0,10);
        display.println(F("Air Monitor IoT"));
        display.println(F("Booting..."));
        display.display();
    }

    // Kết nối Wi-Fi
    WiFi.begin(ssid, password);
    int timeout = 0;
    while (WiFi.status() != WL_CONNECTED && timeout <
20) {
        delay(500); Serial.print("."); timeout++;
    }
}

```



```

Serial.println();
if (WiFi.status() == WL_CONNECTED)
    printLog("WiFi OK! IP: " +
WiFi.localIP().toString());
else
    printLog("WiFi timeout - Offline mode");

// Web Server
server.on("/", handleRoot);
server.begin();

// Blynk
Blynk.config(BLYNK_AUTH_TOKEN);

// Telegram
secured_client.setInsecure();
secured_client.setTimeout(8000);
if (isTelegramEnabled())
    bot.sendMessage(CHAT_ID, "[Wokwi] Bot da khoi
dong!", "");
}

// ----- VÒNG LẶP CHÍNH -----
void loop() {
    Blynk.run();
    server.handleClient();

    // Kiểm tra Telegram
    static unsigned long lastCheck = 0;
    if (millis() - lastCheck >= BOT_CHECK_INTERVAL) {
        lastCheck = millis();
        int n = bot.getUpdates(bot.last_message_received
+ 1);
        // ... xử lý lệnh Telegram ...
    }

    // Đọc cảm biến và cập nhật
    if (millis() - lastUpdate > updateInterval) {
        lastUpdate = millis();
        if (!useSimulatedData) {
            int rawCO = analogRead(PIN_CO_SENSOR);
            coValue    = map(rawCO, 0, 4095, 0, 2000);
            int rawPM = analogRead(PIN_PM_SENSOR);
            pmValue    = map(rawPM, 0, 4095, 0, 150);
        }
        // Cập nhật Blynk
        Blynk.virtualWrite(V1, coValue);
        Blynk.virtualWrite(V2, pmValue);
    }
}

```

```

        // Xác định trạng thái
        int newStatus = 0;
        if (coValue > 1000 || pmValue > 50)
newStatus = 2;
        else if (coValue > 400 || pmValue > 20)
newStatus = 1;

        // Cập nhật OLED
        display.clearDisplay();
        display.setCursor(0,0);
        display.setTextSize(2);
        display.printf("CO: %d\n", coValue);
        display.printf("PM: %d\n", pmValue);
        display.display();

        // Gửi cảnh báo Telegram khi thay đổi trạng thái
        if (newStatus != currentStatus) {
            currentStatus = newStatus;
            if (isTelegramEnabled()) {
                String msg = (currentStatus==0) ? "KHONG KHI
DA TOT!" :
                                (currentStatus==1) ? "CANH BAO:
CHAT LUONG KEM!" :
                                                "BAO DONG
DO! XU LY NGAY!";
                bot.sendMessage(CHAT_ID, msg, "");
            }
        }
        yield();
    }
}

```